



— ARCHITECTURE DESIGN

A fleet that divides instead of resizing.

Orchestrate **2 to a million** tightly-coupled machines by confining coordination to bounded **cells** that split and merge as the swarm grows — no global optimizer, no global consensus.

1_M

TARGET FLEET
SIZE

O(1)

COORD COST /
NODE

2×

SPLIT / MERGE
BAND

1 field

DRIVES
PLACEMENT

01 The requirement that forces the whole design

The brief asks for three things at once: machines that are **tightly coupled**, a fleet that scales **toward one million**, and dispatch that stays **low latency**. Taken literally and flat, that trio is not buildable — each pair fights the third.

CONSTRAINT A

Tight coupling

Machines share state, barriers, or a leader. Coordination has a real per-link cost.

CONSTRAINT B

Scale to 1M

All-to-all is $O(N^2)$ — a million nodes is 10^{12} links. Global consensus caps at ~5–7 voters.

CONSTRAINT C

Low latency

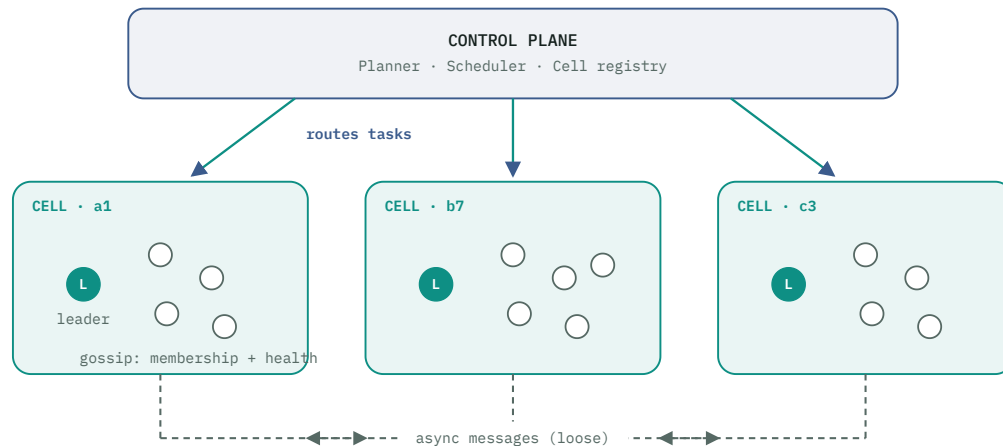
Coordinated global decisions can't also be fast. The speed of light is the ceiling.

A + B + C, applied to one flat topology → **impossible**

The escape hatch every large tightly-coupled system reaches for is the same: **confine the tight coupling to bounded groups, and make the groups loosely coupled to each other.** Those groups are cells. Tight coupling lives *inside* a cell; cells barely talk. Growing from ten thousand machines to a million just means **more cells** — coordination cost per node stays flat. Everything below follows from that one move.

02 Topology

Four parts, and only four: a **control plane** that plans work, **cells** that run it, **gossip** for membership and health inside each cell, and **async messages** between cells. That is the irreducible core — nothing else is required to hit the requirements.



Tight coupling is contained; loose coupling spans the fleet. Inside a cell, a Raft-elected leader coordinates workers and gossip (SWIM) tracks who is alive. Between cells there is only asynchronous message passing — the boundary that keeps the coordination bill flat as cells multiply.

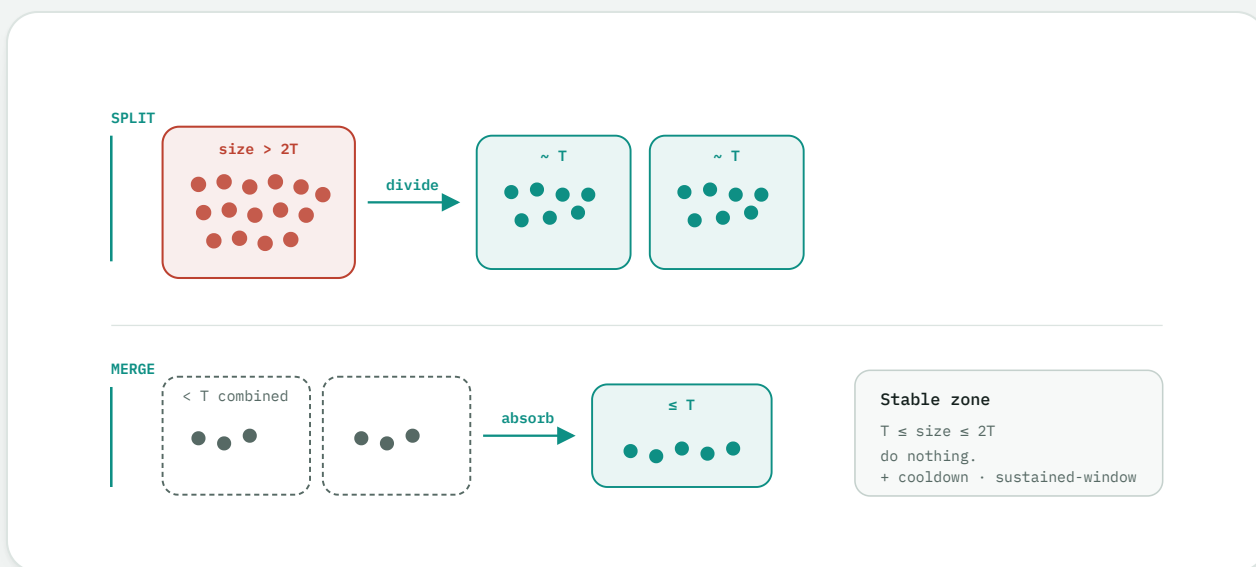
Why gossip inside, a planner on top

Gossip — specifically **SWIM**, the protocol behind Serf, Consul, and Cassandra — is genuinely fast and carries *constant* per-node overhead, but only for the job it's good at: membership, failure detection, and spreading capacity metadata. A new machine joins the swarm by gossiping itself in; no central registration bottleneck. That is your dynamic scaling at the membership layer.

What gossip must *not* own is authoritative task assignment. Tight coordination needs someone to decide — a barrier to release, a leader to elect — so the queue is fed by a **planner**: submitters hand in high-level jobs, the planner decomposes them into tasks and routes them to the right cell, and workers **pull**. Centralized planning, decentralized dispatch — the hot path has no central bottleneck.

03 Mitosis — how cells scale

Cells do not continuously resize to chase an optimum. Rebalancing a tightly-coupled cell means tearing down and rebuilding its coordination state — the single most expensive operation in the system — so you do it rarely, and only at thresholds. A cell has a target size T ; it **splits** when it grows past $2T$ and two neighbors **merge** when together they fall below T . The gap between those two triggers is a $2\times$ hysteresis band, wide enough that a freshly-split cell can never immediately re-merge.



Split at $2T$, merge below T — the $2\times$ gap is what stops thrashing. A cell that just split sits at $\sim T$, far from the merge trigger, so it can't oscillate. Two guardrails harden it further: a **cooldown** (no split/merge on the same cell within minutes) and a **sustained-window** check so a momentary burst of joins doesn't trigger a needless split.

Starting thresholds

Size T off the coupling tightness — the tighter the coordination, the smaller the cell, because barrier latency grows with N and the slowest member gates everyone. Begin with counts; once running, graduate the trigger from a *count* to a *measured* signal (e.g. split when the cell's barrier-completion p99 crosses your latency SLO).

COUPLING	TARGET T	SPLIT >	CELL CHARACTER
● Barrier / lockstep	64–128	128–256	small, one rack/AZ
● Leader-driven	~1,000	2,000	medium
● Message-passing	few×10 ³	2× that	large

R1 **Resize only at task boundaries.** A machine mid-barrier is a member of the barrier group — yanking it corrupts the computation. Splits and merges wait for a safe point, and the leader coordinates the membership change as an explicit step.

R2 **New joins fill, then divide.** Machines gossip in and land in cells until one crosses 2T and splits. That *is* the dynamic scaling — no global optimizer chasing "optimum."

04 The job declares its own coupling

Coupling isn't a global mode — it's a per-job property the submitter declares, exactly like Slurm's MPI job type, Volcano's gang scheduling, or AWS placement groups. One field in the manifest becomes the single input that dials cell size, placement locality, resize-safety, and admission policy all at once.

← TIGHTEST LOOSEST →

BARRIER

Lockstep

small cells • gang

- Synchronous ML (all-reduce SGD)
- CFD / climate simulation
- BSP graph (Pregel supersteps)
- Classic MPI / HPC

MESSAGE-PASSING

Chatty, not blocking

medium cells

- Actor systems
- Async / Hogwild ML + param server
- Streaming pipelines

INDEPENDENT

Embarrassingly parallel

large cells • spray

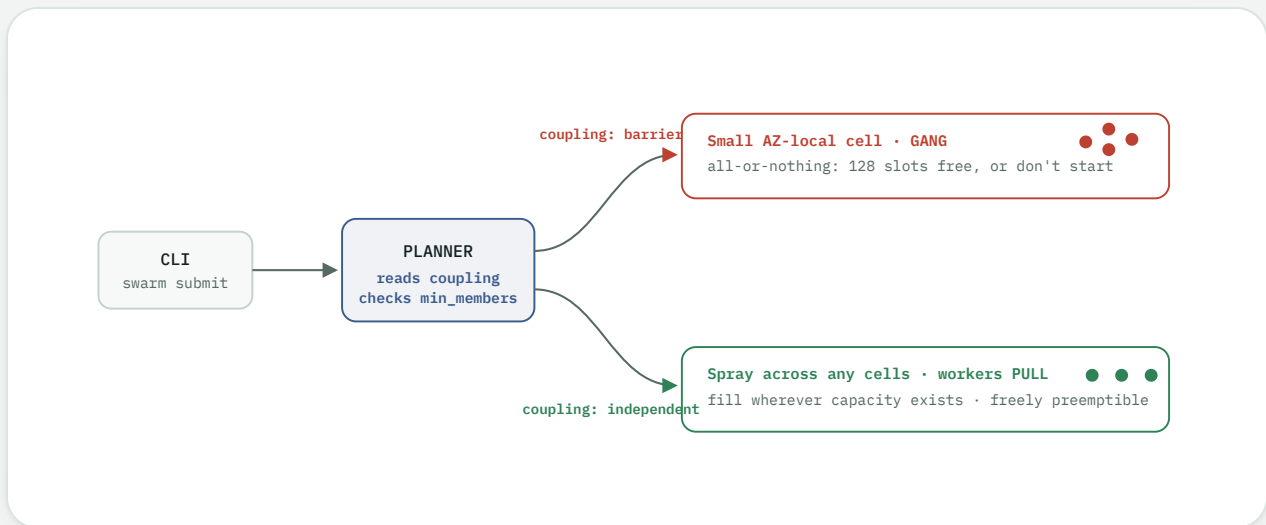
- Frame rendering
- Map-reduce map phase
- Monte Carlo sampling
- Parameter sweeps

What the submitter writes

● ● ● job.yaml

```
coupling: barrier           # independent | message-passing | leader | barrier
min_members: 128           # gang size — all-or-nothing admission
locality: same-az          # affinity hint for tight jobs
```

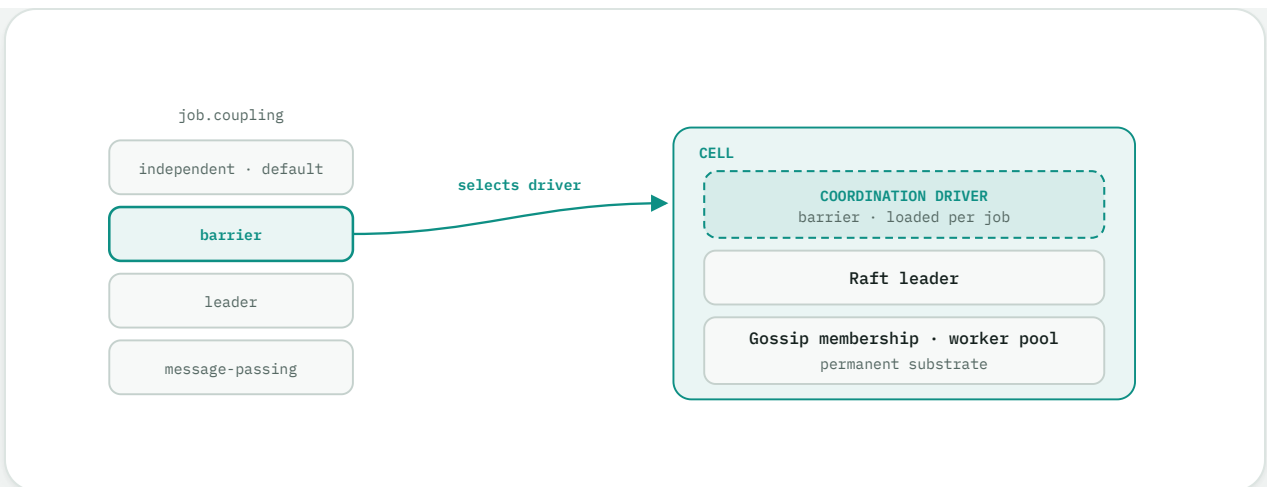
How the planner reads it



One field, three downstream decisions. The coupling declaration routes placement (tight jobs into small, low-latency, AZ-local cells; loose jobs sprayed anywhere), sets resize-safety (barrier jobs are frozen against mid-flight resize), and sets admission — a **gang** job must be placed all-or-nothing, or its started half deadlock waiting on peers that were never scheduled.

05 Substrate stays fixed; the driver plugs in per job

The flavor of coordination belongs to the *job*, not the platform — a Monte Carlo run shares nothing, training needs a barrier, a graph job wants supersteps. So a cell isn't born with a personality. Every cell runs the same **permanent substrate**; what the leader actually executes is a **coordination driver loaded from the job's coupling field** for that job's lifetime. The cell is the room; the job brings its own coordination style and packs it up when it leaves.



The cell is generic; the job supplies its coordination. Gossip, the Raft leader, and the worker pool are identical in every cell. The coupling field selects one of four drivers, which the leader loads for the job's lifetime and unloads when it finishes — so the same cell can run training today and a render tomorrow.

The four drivers differ most in the two things that actually constrain a design: **what a failure costs** and **how big the cell can get**. That pairing is really what a job is choosing between.

DRIVER	WHAT THE LEADER RUNS	FAILURE COST	CELL SIZE	RESIZE
● independent	nothing — a pull queue	trivial · re-queue task	large · 1000s	any time
● barrier	gate each step · all-reduce	high · checkpoint restart	small · 10s–100s	checkpoint only
● leader	assign rounds · hold state	moderate · re-elect leader	medium · ~1000s	between rounds
● message-passing	route mailboxes	low · restart one actor	large · 1000s	any time

R3 **Unspecified coupling defaults to independent.** The safe floor — no coordination, can't deadlock, trivial recovery, resizes freely. Users opt into the tighter drivers explicitly.

R4 **Tightly-coupled jobs get dedicated cells.** A cell runs one flavor at a time, so its leader never juggles a barrier for job A and a router for job B. The gang reserves the cell for its lifetime; independent jobs, needing no driver, share cells freely.

06 The v1 task library

A built-in task isn't a demo — it's the on-ramp. Each template bundles a driver plus sensible defaults (`min_members`, `locality`, how to split the work, how to merge results), so a user runs `swarm run monte-carlo --samples 1e12` without ever thinking about cells. The v1 suite spans all four drivers, so the library doubles as proof every mode works.

TEMPLATE	DRIVER	MIN_MEMBERS	SPLIT BY	MERGE
decrypt Recover a key or plaintext from known ciphertext by testing candidate keys across a range.	 independent	1	keyspace range	first-hit
brute-force Find an unknown input — password, hash preimage — by trying candidates from a wordlist or mask.	 independent	1	candidate space · wordlist / mask	first-hit
monte-carlo Estimate a result from a huge number of independent random samples, then aggregate — pricing, risk, physics.	 independent	1	seed / sample block	sum · statistics
batch-render Produce many independent outputs — 3D frames, ray-traced tiles, video segments — then reassemble.	 independent	1	frame / tile / segment	reassemble
param-sweep Run one program across a grid of configurations — hyperparameter search, ablations, tuning.	 independent	1	config point	rank · collect

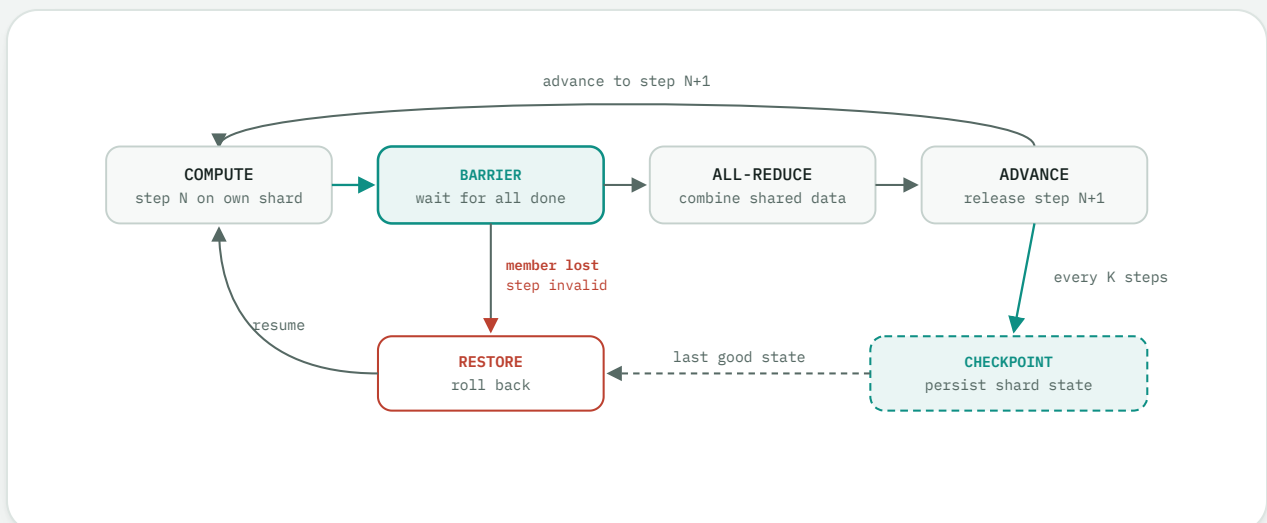
TEMPLATE	DRIVER	MIN_MEMBERS	SPLIT_BY	MERGE
dist-training Train one model across machines, averaging gradients every step (synchronous SGD / all-reduce).	● barrier	8+	data shard	all-reduce gradients
sci-sim Simulate a physical system on a decomposed grid, exchanging boundaries each timestep — CFD, climate, N-body.	● barrier	8+	spatial domain	boundary exchange
agent-sim Simulate many autonomous agents that message their neighbors — epidemiology, traffic, markets.	● message-passing	4+	agent partition	aggregate state
graph-compute Run graph algorithms — PageRank, shortest paths, community detection — in coordinated supersteps.	● leader	4+	vertex partition	superstep combine

Because the default is `independent`, five of the nine need no coordination at all — the bread-and-butter, where most users start. `decrypt` and `brute-force` stay separate: they share the driver but split work differently (a keyspace range vs. a wordlist/mask candidate space) and are documented around the legitimate uses — key recovery, password auditing, cipher benchmarking. The two barrier

templates are the flagship HPC workloads; `agent-sim` and `graph-compute` exercise the remaining two drivers so nothing ships untested.

07 Cell internals: the barrier driver

Barrier is the hardest driver and the one the flagship workloads (`dist-training` , `sci-sim`) depend on, so it's the reference protocol. A barrier cell runs the same substrate as any other — gossip, a Raft leader, a worker pool — but its leader loads a **barrier coordinator** that marches every worker through one strict lockstep loop and owns the checkpoint that makes failure survivable.



Every step is the same four beats, and one checkpoint is the whole safety net. The barrier (accent) is the defining hop — the moment the fast workers stop and wait for the slowest. The red edge is the only failure story a barrier job has: a lost member invalidates the in-flight step, so there is nowhere to resume but the last checkpoint.

The step loop

The coordinator drives one iteration at a time. Every worker computes step N on its own data shard, then reports *done* plus its partial result. The coordinator holds the release until the **last** worker checks in, all-reduces the partials into the shared data (averaged gradients, exchanged boundaries), and only then releases everyone into

step $N+1$. Nothing races ahead; the whole cell is always on the same step number. Every K steps it pauses to persist shard state to durable storage.

The straggler is the tax you always pay

Because the barrier waits for the slowest, one slow machine slows the entire cell *every step* — this is the reason barrier cells stay small (the more members, the higher the odds one is slow, and the longer the tail). The coordinator sets a **per-step deadline**: a worker that misses it isn't waited on indefinitely, it's declared dead by gossip and handled as a failure. That converts an unbounded hang into a bounded, recoverable event.

Failure and recovery

A worker lost mid-step makes that step's partial results incomplete, so the step is **invalid** — there is no partial-credit in lockstep. Recovery is always the same: restore the last checkpoint, re-partition the shard across the surviving (or replacement) members, and replay forward. If the *leader* is the one lost, Raft elects a new leader from within the cell and it resumes from the replicated checkpoint metadata. The checkpoint cadence K is the one real tuning knob: smaller K means less recomputation after a failure but more overhead in the steady state.

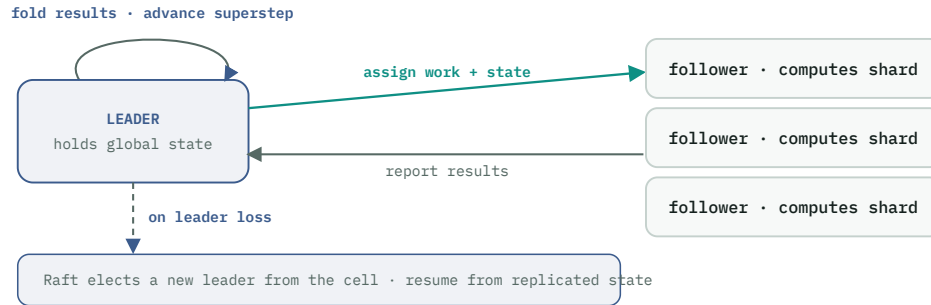
Where mitosis meets lockstep

This is the collision flagged back in section 03 (rule R1), now made concrete. A member can't join or leave mid-step without corrupting the barrier group, so **split and merge happen only at a checkpoint boundary**: the coordinator quiesces at the next checkpoint, persists state, the cell divides, each half reloads its slice of the shard, and both resume from that checkpoint. Growing a running job's membership reshapes the data partition the same way — also a checkpoint-boundary operation, never a live one.

- B1 Bound the barrier with a per-step deadline.** A missed deadline is a failure to recover from, not a wait to endure — otherwise one hung worker stalls the cell forever.
- B2 Checkpoint every K steps.** It's the only resume point a lockstep job has; K trades steady-state overhead against recompute-on-failure.
- B3 Resize only at a checkpoint boundary.** Quiesce, snapshot, divide or grow, reload shards, resume — never swap a member mid-step.
- B4 All members present before step 0.** Gang admission is mandatory here — start with a partial group and it deadlocks at the very first barrier, waiting on peers that were never scheduled.

08 Cell internals: the leader driver

The leader driver runs coordinated **rounds** (BSP supersteps) with one authority holding the global state. It powers `graph-compute`. Unlike barrier, the workers aren't symmetric peers — the leader hands out work and folds results, so its failure story is asymmetric.



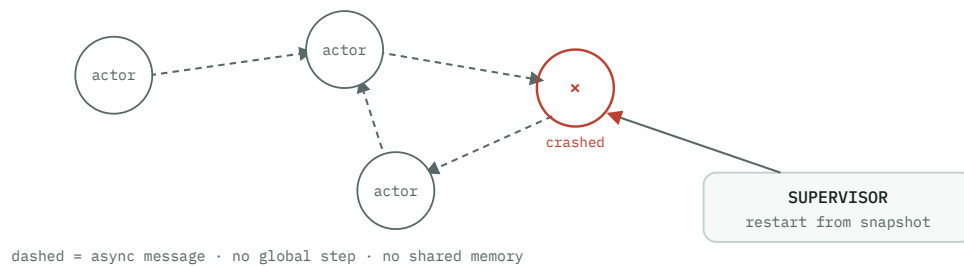
One authority, symmetric rounds, asymmetric failure. A lost follower is a cheap reassignment inside the round; a lost leader is the real event — Raft promotes a new one from the cell and resumes from the replicated checkpoint. The leader's fan-out/fan-in bandwidth is what caps the cell at low thousands.

L1 Only the leader's state is precious. Replicate and checkpoint it; followers hold nothing that can't be recomputed from a reassignment.

L2 Membership changes between rounds. The leader rebalances assignments at a round boundary — cheaper than barrier because the leader mediates the reshape.

09 Cell internals: the message-passing driver

The message-passing driver is the actor model: each worker has an address and a mailbox, sends messages to specific peers, and reacts to what arrives. There is no global barrier, no central authority, and no shared memory — progress is emergent. It powers `agent-sim`.



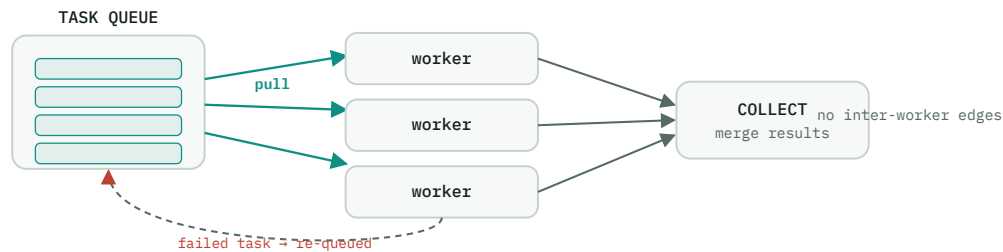
No barrier, no boss — correctness lives in the protocol. Actors make progress independently, so failure is local: a supervisor restarts a crashed actor from its snapshot and in-flight messages are redelivered. The price of that resilience is no global ordering or consistency guarantee — the message protocol has to provide it.

M1 Design messages idempotent and order-tolerant. There is no global ordering; a redelivered or out-of-order message must not corrupt state.

M2 Failure is local. A supervisor restarts one actor from its snapshot — no global rollback, so cells can stay large.

10 Cell internals: the independent driver

The default, and the simplest: no driver at all. The cell is a pull queue, workers grab tasks that reference nothing but their own input, and results are collected at the end. It powers five of the nine library templates — the bread-and-butter of large-scale compute.



The whole coordination story is a queue. Workers never touch each other, so recovery is trivial — a failed task simply goes back on the queue for someone else. This is the one driver where a cell can grow or shrink *mid-job* (the exception to rule R1), because there is no coordination state to disturb.

I1 Make tasks idempotent and re-runnable. Recovery is re-queue-and-retry, so a task that runs twice must produce one correct result.

I2 The only shared step is the final merge. If work needs to coordinate before the end, it isn't independent — it belongs on a heavier driver.

11 The CLI is the operational surface

Operational simplicity is delivered as a thin client over the control-plane API — the whole million-node fleet reduced to a handful of verbs a person actually types.

```
$ swarm submit job.yaml --wait
planned 4,096 tasks · coupling=barrier · gang=128
placed → cell b7 (az-1) · admitted ✓

$ swarm ps
CELLS 812  MACHINES 41,203  JOBS 27  splits/1h 9

$ swarm scale job-4f2 --members +512
$ swarm logs job-4f2 --follow
$ swarm cell b7 --describe  # leader, members, health, size vs 2T
```

PART II · BUILD

12 Tech stack

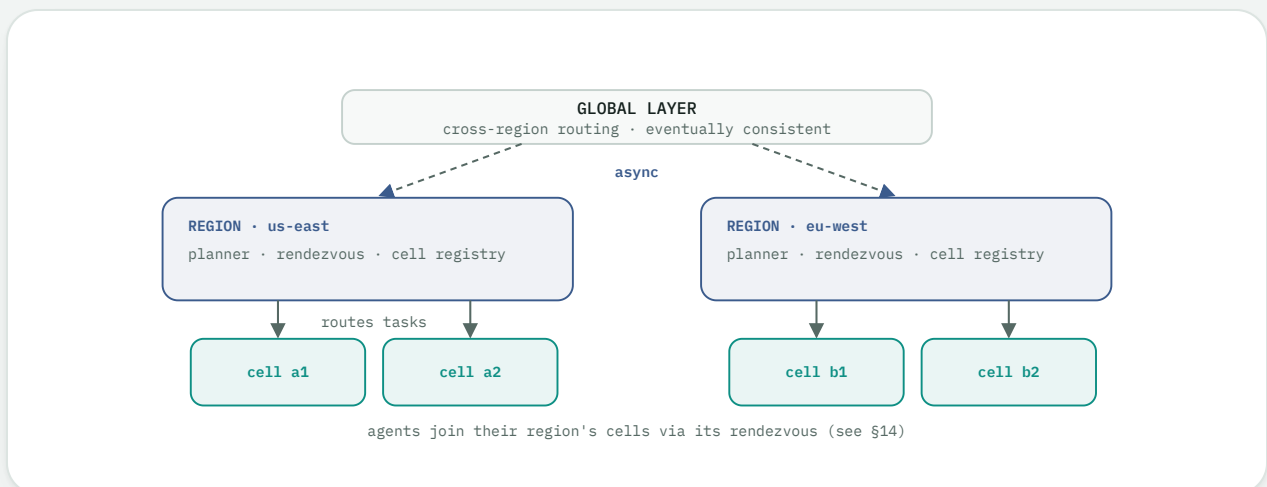
The stance is **reuse proven libraries for the hard distributed primitives, build only what's unique to Swarm**. **Go** for the control plane, agent, and CLI — it's I/O-bound orchestration and the whole primitive ecosystem is already Go-native. **Rust** on the hot path: the WASM host and the worker data path, where throughput and latency matter. Both cross-compile to every OS and architecture, so the agent ships as one native binary per platform.

COMPONENT	TECHNOLOGY	STANCE
Control plane · planner · scheduler	Go	build
Cell agent (runs on every machine)	Go + Rust runtime	build
Membership + failure detection	memberlist · SWIM	reuse
Per-cell leader + replicated log	hashicorp/raft	reuse
RPC · plane↔agent · leader↔worker	gRPC / protobuf	reuse
Cell-to-cell async messages	NATS	reuse
Open-tier workload sandbox	Wasmtime · WASM/WASI	reuse
Core-tier isolation + GPU	cgroups / Job Objects + passthrough	build-thin
Barrier collectives (core tier)	NCCL · custom over gRPC	mixed
Checkpoint + artifact store	pluggable object store · S3/MinIO/local	reuse
Control-plane state · registry · jobs	etcd per region → FoundationDB at scale	reuse
Result verification (open tier)	redundant execution + quorum	build
Observability	OpenTelemetry + cell-level aggregation	reuse
Mitosis · coupling drivers · placement	Go / Rust	build · core

The **build** rows are the Swarm-unique core — the mitosis controller, the four coupling drivers, and capability-based placement. Everything marked **reuse** is a well-worn library we integrate rather than reinvent.

13 The control plane is regional

A single control plane cannot hold a million machines — its state store and scheduler would become the exact bottleneck the cell design exists to avoid. So the control plane **shards by region**, mirroring the fleet: each region owns its own planner, rendezvous, and cell registry, and regions coordinate only **loosely and asynchronously**. A thin global layer handles cross-region routing and an aggregate, eventually-consistent view — never strongly-consistent state on the hot path.



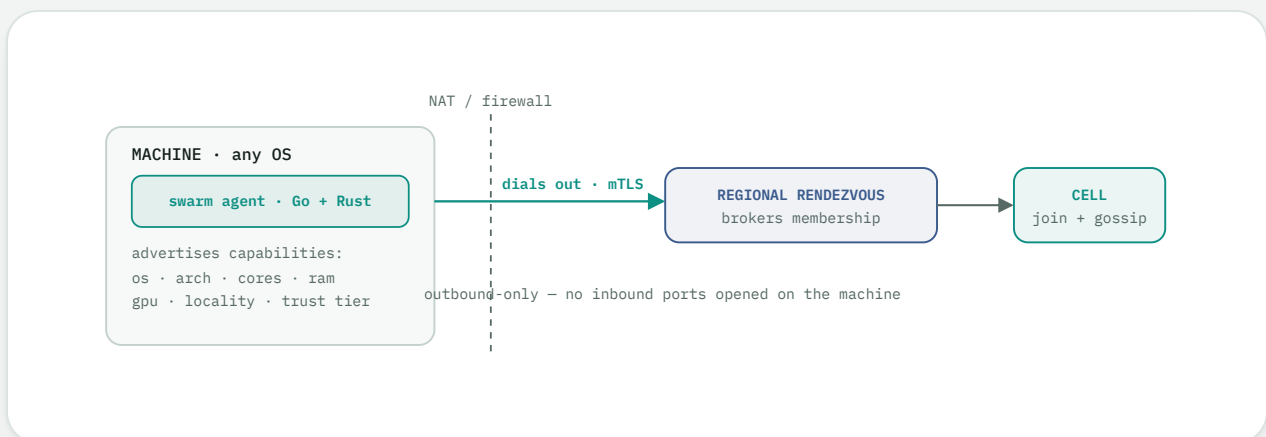
The fleet's shape, one level up. The control plane shards the same way the fleet does — per region, loosely coupled — so no single scheduler or store ever sees all million machines. The global layer routes across regions but holds only an eventually-consistent view.

A1 No global strongly-consistent state on the hot path. A region schedules from its own store; the global layer is eventually consistent and off the critical path.

A2 A region is a blast-radius boundary. A region failing takes its own cells, not the fleet — and its agents re-home to a healthy region.

14 The cross-OS agent and NAT-friendly networking

Swarm installs as **one native binary per OS/arch** (Go + the Rust runtime). Because volunteer and edge machines sit behind NAT and firewalls, the agent **dials out** to its regional rendezvous rather than being dialed into — no inbound ports, works anywhere. On connect it advertises its **capabilities** over gossip, and those capabilities are exactly what the planner places jobs against.

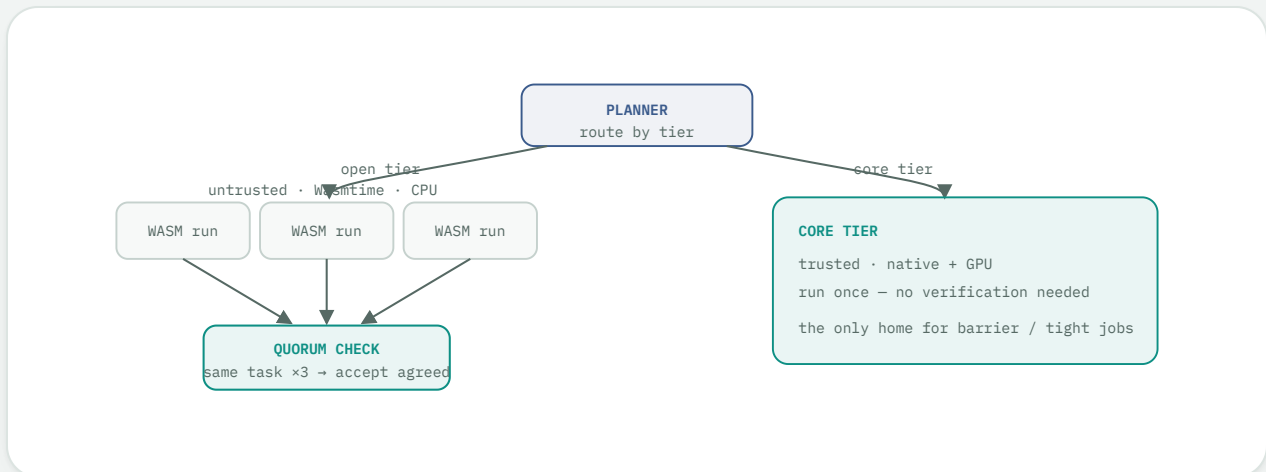


One binary, outbound-only. The agent installs on any OS and dials out to its regional rendezvous, so it works behind NAT and firewalls with no inbound ports. Its advertised capabilities — including which trust tier it belongs to — are what the planner matches jobs against.

15 Two execution tiers

Trust and capability are the same axis (section 05), so the runtime a task gets is decided by the **tier of the machine it lands on**. Untrusted **open-tier** machines run CPU workloads in a **WASM sandbox** — the one sandbox uniform across every OS — and because an untrusted machine can return wrong results, every open-tier result

is confirmed by **quorum across redundant runs**. Trusted **core-tier** machines run native with GPUs, once — the only place barrier training and simulation can go, since WASM has no GPU path.



The machine's tier decides the runtime. Untrusted open-tier machines run CPU workloads in a WASM sandbox, and every result is confirmed by quorum across redundant runs. Trusted core-tier machines run native with GPUs, once — the only place barrier training and simulation can go.

- V1 Open-tier results are quorum-verified before acceptance.** An untrusted machine can lie or fault; run each task redundantly and accept only the agreed answer.
- V2 GPU and tight jobs are core-tier only.** WASM has no production GPU path, so training and simulation run native on trusted machines.
- V3 Sensitive data never leaves the core tier.** Anything confidential stays on trusted infrastructure; the open tier only ever sees non-sensitive, verifiable work.

16 Failure modes and operations

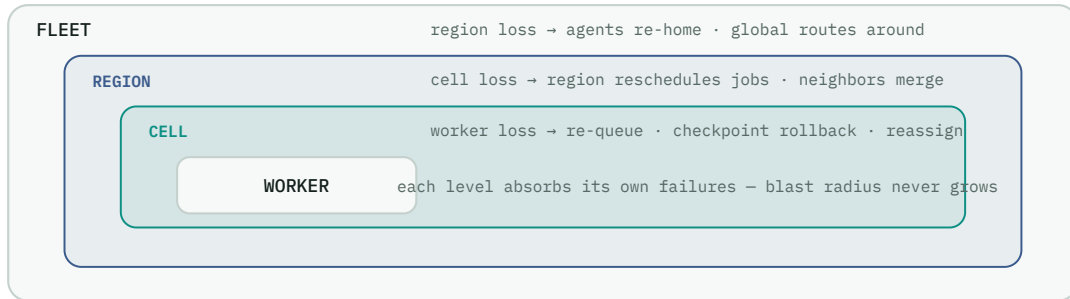
The whole architecture is a set of **containment boundaries**: a failure is absorbed at the level it happens and never escalates outward. A worker dying is a cell's problem; a cell dying is a region's problem; a region dying is the fleet's problem — and each outer level barely notices the inner one.

Self-registration and the reconnect loop

Machines join themselves — the swarm never reaches in. An agent **dials out** to its regional rendezvous and retries with **backoff and jitter, forever**, failing over to another region if its own is unreachable (the jitter prevents a thundering herd when a region recovers). It **enrolls once** for a SPIFFE identity, joins a cell, and stays a member via gossip heartbeats. If the connection drops it simply **resumes the dial loop — keeping its identity** — and the agent is crash-only, so a restart just re-runs the same loop.



Join yourself; never give up. Registration is a loop, not an event: dial with backoff until accepted, enroll once, heartbeat to stay, and on any drop fall straight back into dialing — keeping the identity you already earned.



Blast radius is bounded by design. Every level of the hierarchy is a containment boundary, so no single failure — a machine, a leader, a cell, even a whole region — can cascade into a fleet-wide outage.

FAILURE	DETECTED BY	RESPONSE
Worker / machine dies	gossip · SWIM	driver-specific: re-queue · rollback to checkpoint · reassign · restart actor
Cell leader dies	Raft	re-elect from the cell · resume from replicated log
Whole cell dies	cell heartbeat lost	planner reschedules its jobs · neighbors merge
Rendezvous dies	agent reconnect	agent dials the next rendezvous in its region
Region dies	region heartbeat lost	agents re-home to a healthy region · global routes around
Regional planner dies	replicated · Raft/etcd	standby takes over · in-flight pulls keep running
Cell network partition	SWIM + Raft	minority steps down · barrier stalls → deadline → treated as failure
Straggler	per-step deadline	declared dead, recovered as a failure (B1)
Open-tier bad result	quorum mismatch	reject and re-run (V1)

Operating at fleet scale

Two things change when the fleet is a million machines. **Observability can't scrape per node** — metrics are pre-aggregated at the cell and rolled up per region, so cardinality stays bounded. **Upgrades roll by draining**: the agent is a single binary, so a cell is cordoned, its jobs checkpoint-migrate or drain, and the new version rolls in — gRPC/protobuf keeps mixed versions compatible during the roll. Machine maintenance is the same primitive: cordon, drain, done.

- 01 **Observe at the cell, not the node.** Pre-aggregate per cell and per region; per-node scraping does not survive a million machines.
- 02 **Upgrade and maintain by draining.** Cordon a cell, let its jobs checkpoint-migrate, roll the binary; protocol compatibility covers the version skew mid-roll.
- 03 **Agents self-register and never give up.** Dial-out with backoff and jitter, retry forever, fail over across regions; identity is kept across reconnects.
- 04 **Detection is adaptive by tier; recovery is automatic.** Fast eviction for tight core-tier jobs, patient timeouts for the flaky open tier; the planner reschedules failed jobs itself, no human in the loop.

17 Security model

Every machine that joins gets a **cryptographic identity** from the control-plane CA, built on **SPIFFE/SPIRE**. It presents an enrollment token — operator-issued for core-tier machines, **anonymous self-serve** for the open tier — gets an mTLS certificate, and from then on every connection (rendezvous, planner, cell gossip) is mutually authenticated and encrypted. The trust tier is baked into the identity, so the planner gates placement on it.



No anonymous access. Identity is issued once at enrollment and presented on every connection thereafter, with the trust tier encoded in the certificate — so authentication, encryption, and tier-based placement all ride the same credential.

What a malicious participant can actually do

A participant is only a worker: it pulls a task, runs it, returns a result, and gossips its own health. So a hostile machine's whole repertoire is three moves — **lie about its result, read what it's handed, and fake many identities** (a Sybil attack, below). The architecture shrinks even that: because open-tier jobs are **independent**, workers never talk to each other, and untrusted machines are never placed in tight cells alongside others — so a bad machine has no peer to attack and its **blast radius is its own single task**. Sensitive data never reaches it (V3), and gossip is authenticated, so it can only ever speak for itself. That is why the threat model is pragmatic: the real risk is a machine lying about its own work, possibly at scale.

Sybil resistance without accounts

A **Sybil attack** is one attacker spinning up many fake identities to overpower a system that assumes one identity is one participant — here, to win a quorum vote and force a wrong result. You cannot stop it with self-reported hardware IDs: a **MAC address** is trivially spoofed, invisible across the internet, and fresh per VM or container — anything an untrusted machine says about itself can be forged. Real hardware binding needs **TPM / secure-enclave attestation** (via SPIRE node attestation), which we use on the core tier but can't rely on for random open-tier machines. So the open tier doesn't try to enforce one-identity-per-machine — it makes extra identities *useless*:

Q **Quorum + random assignment.** Each task runs on K machines chosen randomly from the whole pool; forcing a wrong answer needs a large fraction of total compute, not a few fake nodes.

R **Reputation gating.** A new anonymous identity starts at zero trust and is always cross-checked; trust is earned only by agreeing with quorum over time — so every Sybil must independently do honest work.

H **Honeypot spot-checks.** Occasionally hand out a task whose answer is known; a lie gets the identity blacklisted. No account required.

P **Optional proof-of-work to enroll.** A small puzzle puts a cost floor under mass identity creation.

Threats and mitigations

THREAT	MITIGATION
Malicious / faulty host lies about results	quorum verification (V1) · sensitive work stays core-tier
Sybil — one attacker, many fake identities	quorum + random assignment · reputation gating · honeypots · optional proof-of-work
Malicious workload code	WASM sandbox (open tier) · signed workloads verified before run · least-privilege WASI
Network attacker (MITM / eavesdrop)	mTLS everywhere · encrypted gossip and NATS
Rogue machine joining	enrollment tokens · issued identity · trust-tier gating
Data exfiltration	sensitive data core-tier only (V3) · encryption at rest and in transit
Cross-tenant interference	dedicated cells per tight job (R4) · sandbox isolation

S1 Every participant has a cryptographic identity. mTLS on every connection; no anonymous access to the control plane.

S2 Workloads are signed and verified before running. An agent refuses code whose signature doesn't check — blocks tampered or spoofed tasks.

S3 Least privilege by default. WASI grants only the resources a job declares; core-tier native jobs run under OS limits.

S4 **Bind identity to hardware only where you can.** Core-tier machines use TPM / SPIRE node attestation; the open tier can't (self-reported IDs are forgeable), so it relies on quorum and reputation instead.

18 **Deliberately not building this**

The core is four moving parts. Everything below is a tempting complication that the threshold-and-declaration model makes unnecessary — each one buys cost and fragility without buying the requirements.

Continuous rebalancing

A control loop chasing "optimum" cell sizes pays the tear-down cost constantly. Split/merge at thresholds gets stability for free.

Cross-cell consensus

Any global agreement reintroduces the $O(N^2)$ / 5–7-voter wall. Cells coordinate only through async messages, never a shared vote.

Placement optimization

Elaborate bin-packing across a million nodes. The coupling field plus "fill until 2T" is enough; measure before adding a solver.

19 **Phased roadmap**

The sequence proves a working, geo-distributed swarm on easy workloads first, then adds capability, openness, scale, and finally self-tuning — each phase gated by a concrete exit criterion. It stays inside the non-goals of section 18: the later phases refine what we designed, never a continuous-rebalancing optimizer or a heavy placement solver.

P0 Core loop + easy mitosis SINGLE REGION · TRUSTED · NATIVE

Agent self-registration (dial-out, backoff/jitter) · gossip membership · split/merge in the independent case · planner → queue → pull · CLI. Templates: keyspace-search, monte-carlo.

Exit — an independent job runs on a self-scaling single-region fleet and returns correct results; cells split and merge under load.

P1 Multi-region reach STILL INDEPENDENT

Regional control planes · per-region rendezvous · retry-forever + cross-region failover · thin global routing · cross-region job spread. Plus the throwaway barrier spike.

Exit — the fleet spans two or more regions, a job spreads across them, agents fail over between regions, and barrier mechanics are proven in the spike.

P2 Tight coupling + full driver set CORE TIER

Per-cell Raft leader · barrier, leader, message-passing drivers · checkpointing · mitosis-at-checkpoint (B3) · gang scheduling · GPU · adaptive-by-tier detection. Templates: dist-training, sci-sim, graph-compute, agent-sim.

Exit — all nine templates run on the core tier; a GPU training job completes with checkpoint recovery after an induced failure; a barrier cell splits cleanly at a checkpoint boundary.

P3 Open / untrusted tier

WASM (Wasmtime) sandbox · result verification (quorum + random assignment) · reputation gating · honeypot spot-checks · anonymous SPIFFE/SPIRE enrollment · workload signing.

Exit — untrusted machines run independent jobs; quorum rejects an injected wrong result; a honeypot-caught liar is de-weighted; anonymous enrollment issues a valid identity.

P4 Scale infrastructure

FoundationDB registry · observability aggregation (cell/region rollup) · rolling upgrades by drain · backpressure / rate-limiting.

Exit — control plane and registry sustain target load with no single bottleneck; a rolling upgrade completes with zero job loss; metric cardinality stays bounded at fleet scale.

P5 Production readiness / GA

Multi-tenancy (quotas, fairness, isolation) · disaster recovery + chaos testing · SLAs and alerting · optional TPM/attestation on the core tier · security audit.

Exit — tenant quotas and fairness are enforced; a DR drill recovers a lost region; the security review passes; SLA targets are met in staging.

P6 Self-tuning + full-scale GA FULL DESIGN REALIZED

Signal-based adaptive mitosis (split on measured p99, the §03 upgrade) · capability/locality-aware placement refinement (short of a solver, per §18) · reputation maturity · sustained 1M-node validation.

Exit — mitosis splits on a measured signal, placement honors capability and locality, and a sustained run at ~1M nodes is stable. The design is realized.

Swarm · cell-based orchestration · design brief

core: cells · gossip · per-cell leader · async messages